

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

ПОЯСНЮВАЛЬНА ЗАПИСКА

до кваліфікаційної роботи освітнього ступеня «бакалавр»
за спеціальністю 121 «Інженерія програмного забезпечення»
(освітня програма «Інженерія програмного забезпечення»)
на тему:

«Вебсервіс з проведення турнірних змагань»

Виконав студент групи ІПЗ-22-3
МУРАВИЦЬКИЙ Владислав Юрійович

Керівник роботи:
ТОЛСТОЙ Ігор Анатолійович

Рецензент:
ВАКАЛЮК Тетяна Анатоліївна

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

«ЗАТВЕРДЖУЮ»

Зав. кафедри інженерії програмного
забезпечення

Тетяна Вакалюк

«__» лютого 2026 р.

ЗАВДАННЯ
на кваліфікаційну роботу

Здобувач вищої освіти: **МУРАВИЦЬКИЙ Владислав Юрійович**

Керівник роботи: **ТОЛСТОЙ Ігор Анатолійович**

Тема роботи: **«Вебсервіс з проведення турнірних змагань»**,
затверджена Наказом закладу вищої освіти від **«__» _____ 2026 р., №__**

Вихідні дані для роботи: **зазначається предмет і об'єкт дослідження**

Консультанти з бакалаврської кваліфікаційної роботи із зазначенням розділів, що їх
стосуються:

Розділ	Консультант	Завдання видав	Завдання прийняв
1	Толстой І.А.		
2	Толстой І.А.		
3	Толстой І.А.		

РЕФЕРАТ

Пояснювальна записка до переддипломної практики містить текстову частину, викладену на ... сторінці друкованого тексту. Список використаних джерел містить 34 найменування і займає ... сторінки. У роботі наведено . ілюстрацій, . таблиці та . додаток. Загальний обсяг роботи становить 33 сторінки

					ІПЗ.КР.Б – 121 – 26 – ПЗ				
Змн.	Арк.	№ докум.	Підпис	Дата					
Розроб.		Муравицький В. Ю.			Вебсервіс для проведення турнірних змагань		Літ.	Арк.	Аркушів
Перевір.								3	XX
Керівник							Житомирська політехніка, група ІПЗ-22-3		
Н. контр.									
Зав. каф.									

ЗМІСТ

ЗМІСТ	4
ВСТУП.....	5
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ВИБІР ТЕХНОЛОГІЧНИХ РІШЕНЬ ДЛЯ АВТОМАТИЗАЦІЇ ТУРНІРНИХ ЗМАГАНЬ.....	7
1.1. Постановка задачі та аналіз бізнес-процесів організації змагань	7
1.2. Аналіз аналогів програмного продукту	9
1.3. Вибір архітектури веб-орієнтованої системи управління турнірами	13
1.4. Обґрунтування вибору інструментальних засобів та вимоги до апаратного забезпечення	16
Висновки до першого розділу.....	21
РОЗДІЛ 2. ПРОЄКТУВАННЯ СИСТЕМИ.....	23
2.1. Визначення варіантів використання та об'єктно-орієнтованої структури системи	23
2.2. Розробка бази даних системи.....	28
ВИСНОВКИ.....	34
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	35
ДОДАТКИ.....	38

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		4

ВСТУП

Актуальність теми. Сучасний етап розвитку спортивної індустрії характеризується глобальною цифровізацією управлінських процесів. Перехід від традиційного паперового документообігу та використання розрізнених електронних таблиць до централізованих веб-платформ зумовлений потребою в мінімізації людського фактора, забезпеченні прозорості результатів та миттєвій обробці великих масивів інформації. Специфіка проведення змагань вимагає оперативного керування даними про учасників, гнучкого налаштування категорій та точного виконання алгоритмів жеребкування.

Цифровізація дозволяє інтегрувати адміністративні функції, такі як реєстрація учасників, ведення рейтингів та фіксація результатів, у єдину систему. Існуючі програмні рішення часто мають закриту архітектуру або обмежені можливості адаптації під специфічні регламенти окремих спортивних федерацій. Особливої ваги набуває проблема забезпечення цілісності та синхронізації даних у режимі реального часу, коли декілька суддівських бригад одночасно фіксують хід поєдинків на різних майданчиках. Також критично важливим є створення універсальних алгоритмів для генерації турнірних структур різних форматів – від олімпійської системи та кругових турнірів до складних систем із втішними поєдинками. Відтак, розробка спеціалізованого веб-сервісу на базі сучасних фреймворків є актуальним інженерним завданням.

Мета роботи – розробка алгоритмів та реалізація програмного комплексу веб-сервісу для автоматизації процесів організації та проведення турнірних змагань.

Завдання роботи. Встановлена мета обумовлює вирішення наступних завдань:

- провести системний аналіз задач управління турнірами та напрямків їх автоматизації;
- визначити архітектуру та узагальнену структуру системи; – обґрунтувати та обрати засоби реалізації програмного комплексу;

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		5

- розробити інформаційне забезпечення та структуру бази даних; – спроектувати алгоритми автоматичної генерації турнірних сіток різних форматів;
- реалізувати підсистему комунікації у реальному часі та модулі автоматизованої звітності;
- виконати програмну реалізацію компонентів веб-сервісу.

Об’єктом дослідження є автоматизація процесів планування та управління спортивними турнірними змаганнями.

Предметом дослідження є методи, алгоритми та програмні інструменти розробки веб-орієнтованих систем для турнірного менеджменту.

Методи дослідження. У роботі використано методи системного аналізу, принципи об’єктно-орієнтованого проектування, теорію реляційних баз даних та сучасні технології веб-програмування.

Практичне значення одержаних результатів полягає у створенні інструментарію для спортивних клубів та федерацій, що забезпечує автоматизацію реєстрації, безпомилкове жеребкування та ведення протоколів у реальному часі. Впровадження системи дозволяє нівелювати помилки при формуванні категорій та значно скоротити час на підготовку стартових протоколів.

Структура та обсяг проекту. Робота складається з вступу, 3-х розділів, висновків, списку використаних джерел із _ найменувань (_ стор.), _ додатка (_ стор.), _ таблиць та _ ілюстрацій. Повний обсяг роботи становить _ сторінки, з них _ сторінок основного тексту.

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		6

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ВИБІР ТЕХНОЛОГІЧНИХ РІШЕНЬ ДЛЯ АВТОМАТИЗАЦІЇ ТУРНІРНИХ ЗМАГАНЬ

1.1. Постановка задачі та аналіз бізнес-процесів організації змагань

Організація та проведення спортивних турнірних змагань із контактних видів спорту та бойових мистецтв (карате, дзюдо, тхеквондо, грепплінг) є складним організаційним процесом, що вимагає чіткої координації дій учасників, тренерів, суддівських бригад та організаторів. Аналіз предметної області дозволяє виділити три ключові етапи життєвого циклу типового турніру: підготовчий (реєстрація та конфігурація), етап жеребкування (генерація сіток) та активний етап (проведення поєдинків у реальному часі). Кожен із цих етапів супроводжується специфічними бізнес-процесами, які потребують автоматизації.

Першим критичним процесом є конфігурація змагань та формування категорій. На відміну від лінійних видів спорту, єдиноборства вимагають жорсткої типізації учасників для забезпечення чесної конкуренції та безпеки. Категорії формуються на основі багатовимірної матриці параметрів: вікових обмежень, вагових лімітів, статі, а також рівня кваліфікації. Окремо виділяються командні дисципліни, такі як естафети або змішані командні змагання, котрі вимагають обробки групових заявок від спортивних клубів з урахуванням квот за статтю та віком [2]. Проблема існуючих методів обліку (зокрема, використання електронних та / або паперових таблиць) полягає у відсутності автоматичної валідації: помилка при внесенні ваги учасника призводить до незворотних збоїв на етапі жеребкування.

Наступним і математично найскладнішим процесом є жеребкування та генерація турнірних сіток. Залежно від регламенту конкретної федерації та кількості учасників у категорії, система повинна підтримувати кілька базових алгоритмічних моделей:

1. Олімпійська система на вибування (Single Elimination) – класичний формат, при якому учасник вибуває після першої поразки. Ключовим алгоритмічним викликом тут є розрахунок сітки для кількості учасників, що не дорівнює

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		7

ступеню двійки (2^n). Алгоритм має автоматично визначати найближчий більший ступінь двійки та розраховувати необхідну кількість «вільних проходів» (BYE), автоматично просуваючи найсильніших учасників (з найвищим посівом) у наступний раунд [3].

2. Система з втішними поєдинками (Double Elimination / Repêchage) – формат, характерний для міжнародних правил, де учасники, які програли фіналістам турніру, отримують право змагатися у додатковій (нижній) сітці за бронзову нагороду [5]. Це вимагає складного реляційного проектування бази даних, оскільки результати матчів основної сітки динамічно формують склад учасників втішної сітки.
3. Кругова система (Round Robin) – формат для категорій із малою кількістю учасників, де кожен змагається з кожним. Алгоритм має генерувати розклад поєдинків та автоматично підраховувати бали за перемоги, нічиї або поразки для формування фінального рейтингу [6].

Додатковою вимогою до алгоритму жеребкування є розведення спортсменів з одного спортивного клубу (Club Seeding) на ранніх етапах змагань, щоб уникнути поєдинків між одноклубниками до фінальних чи півфінальних раундів [7].

Третій ключовий процес – управління турніром у режимі реального часу. Під час змагань поєдинки проходять паралельно на кількох змагальних майданчиках (татамі). Суддівські бригади використовують електронні пристрої для фіксації балів, попереджень та підсумкових результатів матчу. З інженерної точки зору це створює проблему конкурентного доступу до бази даних. Необхідно забезпечити транзакційну цілісність даних: результат матчу має миттєво фіксуватися в системі, автоматично переводити переможця у наступний раунд сітки та оновлювати інформацію на публічних табло без необхідності оновлення сторінки клієнтом [4].

На основі проведеного аналізу бізнес-процесів формалізовано перелік задач, які мають бути вирішені під час розробки програмного комплексу.

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		8

Система повинна забезпечити:

- гнучке управління ролями користувачів із розмежуванням прав доступу;
- створення конструктора турнірів та категорій із системою автоматичної валідації вхідних даних при подачі заявок;
- розробку алгоритмічного ядра на стороні сервера для автоматичної генерації турнірних дерев за описаними системами;
- проектування нормалізованої реляційної бази даних, здатної підтримувати рекурсивні зв'язки між матчами;
- розробку REST API для забезпечення взаємодії між сервером, панелями суддів та публічним інтерфейсом користувача;
- інтеграцію технологій двонаправленого зв'язку (WebSockets) для миттєвої синхронізації результатів поєдинків;
- автоматизацію документообігу, включаючи генерацію стартових протоколів та підсумкових звітів.

Розв'язання цих задач дозволить створити єдиний інформаційний простір для проведення змагань, що мінімізує вплив людського фактора, прискорює процес обробки даних та забезпечує прозорість суддівства на всіх етапах турніру.

1.2. Аналіз аналогів програмного продукту

Для визначення функціональних вимог та проектування архітектури системи необхідно провести порівняльний аналіз існуючих програмних рішень для управління спортивними заходами. Ринок спеціалізованого програмного забезпечення (ПЗ) представлений як універсальними платформами, так і вузькоспеціалізованими сервісами.

Система MyUventex є одним із лідерів північноамериканського ринку автоматизації змагань із бойових мистецтв. Платформа забезпечує повний цикл управління заходом: від онлайн-реєстрації та прийому платежів до генерації сіток і ведення протоколів. До переваг системи належать високий рівень автоматизації

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						9
Змн.	Арк.	№ докум.	Підпис	Дата		

рутинних процесів реєстрації, гнучкий конструктор категорій на основі комбінації параметрів віку, ваги та досвіду, а також наявність спеціалізованого модуля для суддівських бригад. Проте існують суттєві недоліки: закрита архітектура та відсутність відкритого API для інтеграції зі сторонніми сервісами, висока вартість ліцензування для невеликих організацій та перевантажений інтерфейс адміністративної панелі, що ускладнює швидке навчання персоналу.

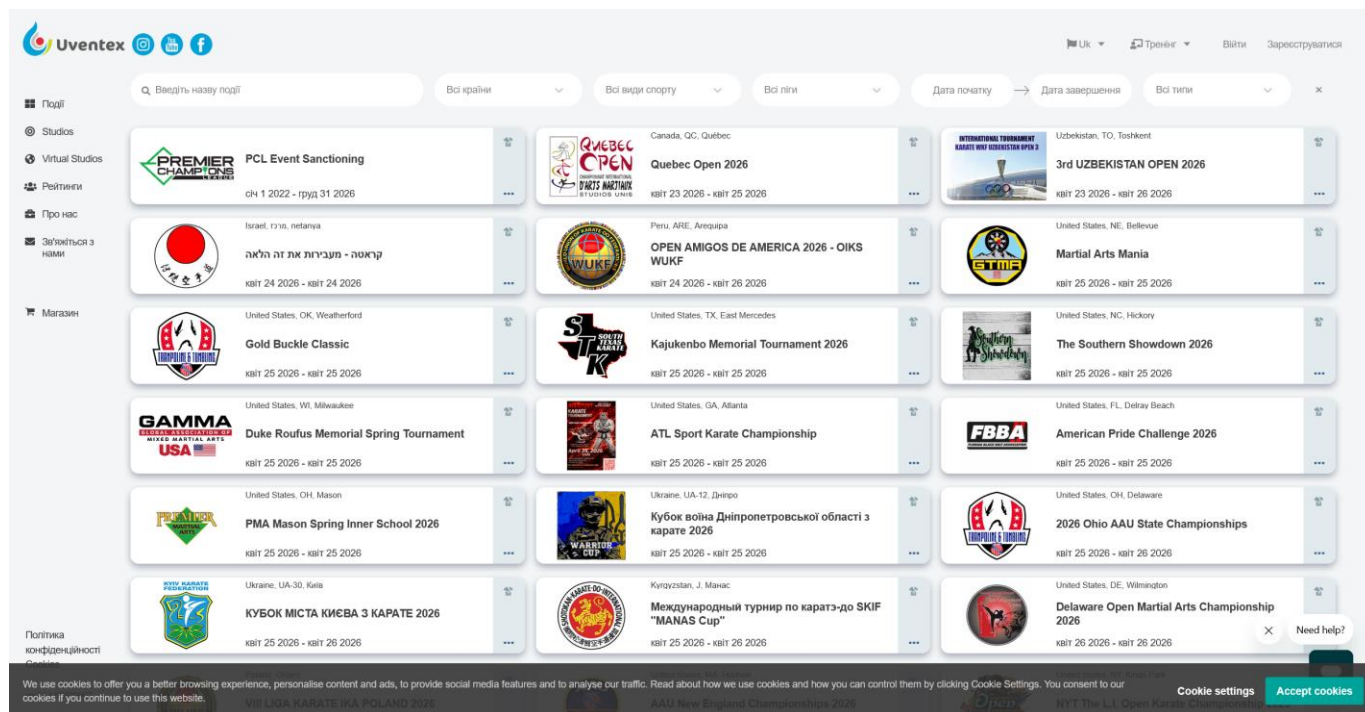


Рис. 1.1. Інтерфейс платформи MyUventex [8]

Платформа Smoothcomp. Даний продукт є сучасним SaaS-рішенням, що спеціалізується на контактних видах спорту. Платформа вирізняється високим рівнем візуалізації даних та інтеграцією з глобальними рейтингами спортсменів. Її перевагами є сучасний та адаптивний інтерфейс користувача, потужна система відображення турнірних сіток із візуалізацією прогресу учасників у реальному часі, а також автоматизована система синхронізації результатів між пристроями суддів та публічним табло. Водночас система орієнтована переважно на західні платіжні системи, має жорстку прив'язку до власних алгоритмів рейтингування, котрі важко адаптувати під внутрішні регламенти федерацій,

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		10

та передбачає високу комісійну плату за кожного зареєстрованого учасника.

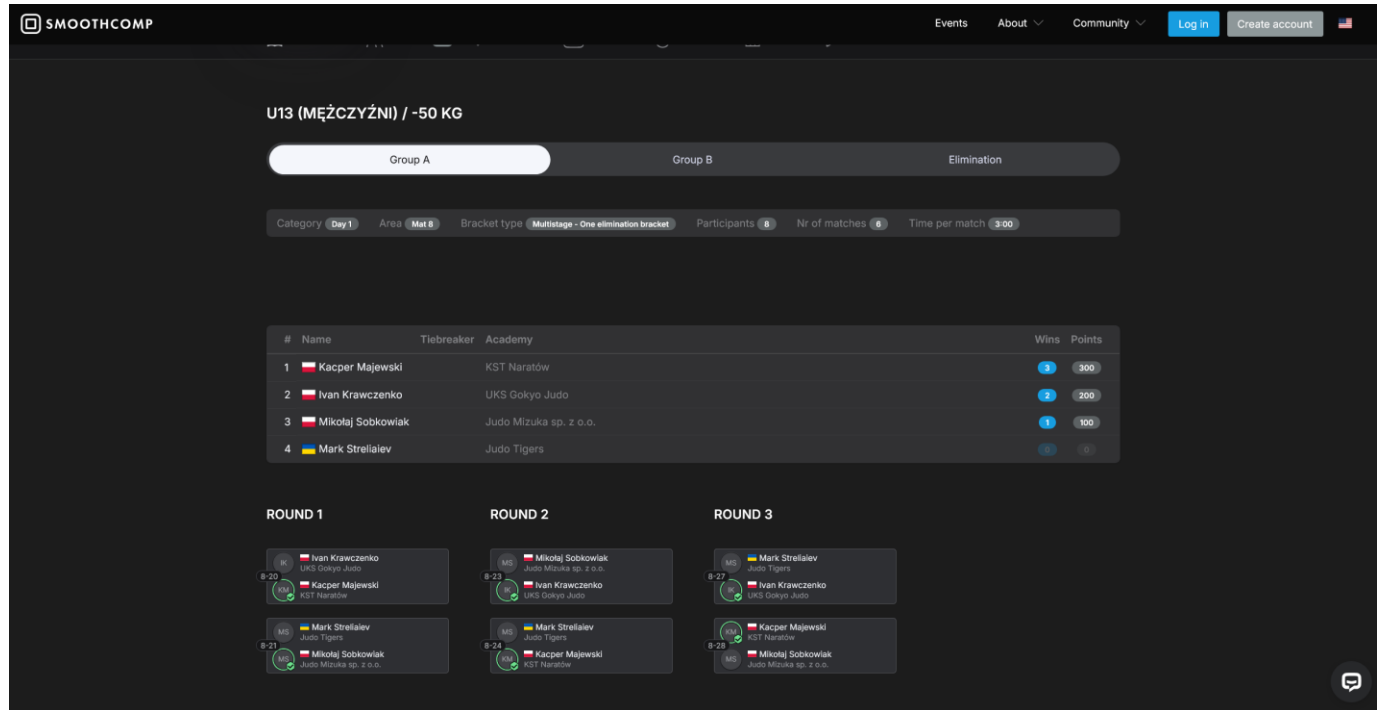


Рис. 1.2. Візуалізація турнірної сітки на платформі Smoothcomp [9]

Система Sportdata є найбільш розгалуженим та функціонально наповненим рішенням, котре підтримує роботу як у хмарному режимі, так і локально. Вона широко використовується великими міжнародними організаціями завдяки своїй стабільності. Перевагами системи є можливість роботи в автономному режимі (offline) без стабільного інтернет-з'єднання, підтримка складних форматів жеребкування та втішних поєдинків, а також наявність окремих модулів для керування професійними табло. Недоліками продукту є морально застарілий інтерфейс, висока складність розгортання та налаштування локальних серверів, що вимагає спеціалізованої технічної підготовки, а також складність масштабування через монолітну структуру ПЗ.

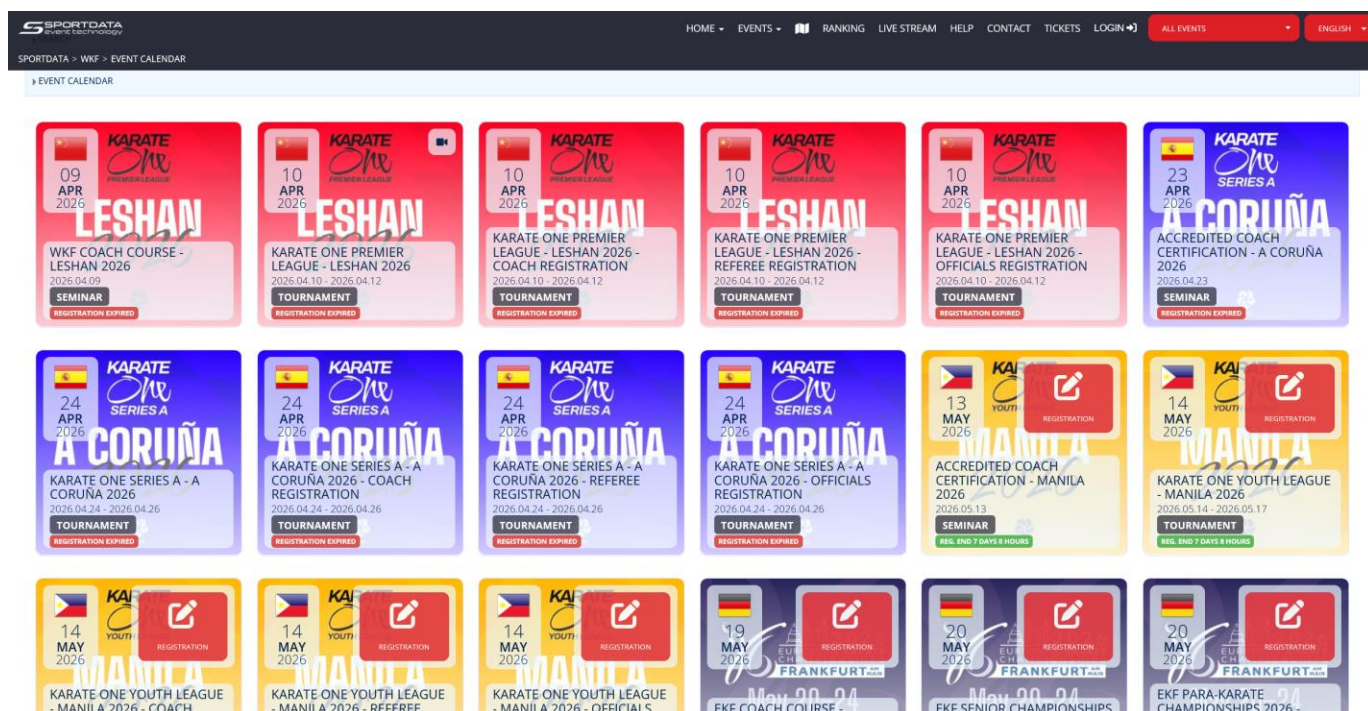


Рис. 1.3. Платформа Sportdata [10]

Для систематизації результатів аналізу основні параметри розглянутих систем наведено у таблиці 1.1.

Таблиця 1.1 – Порівняльний аналіз аналогів турнірних платформ

Функціональна можливість	MyUventex	Smoothcomp	Sportdata
Сучасність інтерфейсу (UI/UX)	Середня	Висока	Низька
Наявність відкритого API	Ні (Закрита система)	Обмежена	Ні (Закрита система)
Модель розгортання	SaaS (Хмара)	SaaS (Хмара)	Гібридна (Хмара + Локально)
Метод синхронізації (Судді-Табло)	Real-time (Пропріетарний)	Real-time (Пропріетарний)	Батчева (Локальний сервер)
Гнучкість регламентів	Висока (в межах системи)	Середня	Висока (Стандарти WKF)
Складність інтеграції для нових клубів	Середня	Низька	Висока

Проведений аналіз свідчить, що більшість існуючих рішень мають закритий програмний код та не надають можливості інтеграції сторонніх модулів через відкриті інтерфейси. Це обмежує можливість кастомізації системи під потреби конкретного замовника.

На основі виявлених недоліків аналогів у таблиці 1.2 сформовано перелік функціональних можливостей та нововведень, котрі будуть реалізовані у власній розробці.

Таблиця 1.2 – Функціональні нововведення та відмінності власної розробки

Функціональна вимога	Обмеження існуючих аналогів	Рішення у власній розробці
Архітектура та Інтеграція	Закриті екосистеми; неможливість підключити власні модулі, сторонні табло чи мобільні додатки.	Повноцінний REST API на базі Django, що дозволяє підключати будь-які зовнішні клієнти.
Алгоритмічна база	Висока гнучкість, але лише в межах наперед затверджених міжнародних регламентів.	Можливість додавати кастомні локальні правила (наприклад, специфічні командні естафети) без переписування системи.
Синхронізація (Real-time)	Прив'язка до дорогих підписок платформи або необхідність розгортання складних локальних серверів.	Використання WebSockets (Django Channels) для миттєвої передачі балів на будь-який пристрій з браузером.
Розгортання та DevOps	Повна залежність від хмарної інфраструктури розробника ПЗ або складне ручне налаштування.	Використання Docker та CI/CD пайплайнів для швидкого розгортання на будь-якому сервері спортивної федерації.

Таким чином, розроблювана система поєднуватиме такі сучасні технологічні рішення, як Real-time синхронізація та API-first підхід із гнучкістю налаштувань, що дозволить використовувати її для широкого спектра спортивних дисциплін.

1.3. Вибір архітектури веб-орієнтованої системи управління турнірами

Проектування програмного комплексу для автоматизації турнірних змагань вимагає вирішення комплексної інженерної задачі: забезпечення транзакційної цілісності даних при паралельній роботі десятків суддівських бригад та підтримки мінімальних мережевих затримок для оновлення результатів у реальному часі. Вибір базового архітектурного стилю безпосередньо впливає на можливості масштабування, складність розгортання та загальну стійкість системи до відмов. У сучасній інженерній практиці для реалізації серверної частини найчастіше розглядають мікросервісну архітектуру та різновиди монолітних систем, кожна з яких має свої переваги та обмеження в контексті спортивного менеджменту.

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		13

Мікросервісний підхід передбачає декомпозицію системи на набір дрібних, незалежних сервісів, кожен з яких виконується у власному процесі операційної системи та комунікує з іншими через легковагові мережеві протоколи [11]. З апаратної точки зору це дозволяє незалежно масштабувати найбільш навантажені вузли. Проте для предметної області контактних єдиноборств впровадження мікросервісів створює низку критичних проблем. Розщеплення бази даних вимагає реалізації складних механізмів розподілених транзакцій. Наприклад, якщо під час поєдинку система має оновити статус матчу, нарахувати рейтингові бали спортсмену та перевести його в наступний раунд рекурсивної турнірної сітки, ці операції повинні виконуватися суворо атомарно. У розподіленій системі найменша мережева затримка або часткова відмова одного з вузлів може призвести до неконсистентності турнірного дерева. Емпіричний досвід індустрії свідчить, що розробка складних систем з нуля у вигляді мікросервісів часто призводить до надмірних витрат на інфраструктуру, тому доцільнішим є підхід початкової розробки моноліту з подальшим виокремленням сервісів виключно за об'єктивної потреби [12].

Зважаючи на описані ризики та необхідність підтримки високої надійності обробки результатів, як архітектурну базу серверної частини обрано патерн модульного моноліту. Цей підхід визначається як архітектурний стиль, за якого система проєктується та розгортається як єдиний обчислювальний вузол, але внутрішньо розділена на суворо ізольовані модулі, що взаємодіють виключно через чітко визначені програмні інтерфейси [13]. На відміну від класичного моноліту із сильною зв'язністю коду, логічна ізоляція дозволяє уникнути непередбачуваного впливу змін в одному модулі на інші компоненти. Водночас відсутність мережевих запитів для міжмодульної комунікації кардинально знижує затримки та дозволяє використовувати надійні локальні ACID-транзакції реляційної бази даних [14].

Взаємодія між серверною частиною та різнотипними клієнтськими застосунками, такими як панель адміністратора, суддівські планшети та публічні табло, базується на строгій клієнт-серверній моделі. Таке розділення дозволяє

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		14

користувачькому інтерфейсу та бізнес-логіці розвиватися незалежно одне від одного [15]. Для виконання стандартних операцій, зокрема реєстрації клубів чи конфігурації категорій, використовується архітектурний стиль REST. Безстанові HTTP-запити у форматі JSON дозволяють серверу легко обробляти паралельні звернення від різних користувачів, не зберігаючи контекст їхніх попередніх дій.

Однак синхронна природа HTTP-протоколу не дозволяє ефективно реалізувати трансляцію поєдинків у режимі реального часу. Використання періодичних опитувань сервера (polling) неминуче призводить до вичерпання пулу з'єднань та перевантаження бази даних. Для вирішення цієї специфічної задачі архітектуру розширено протоколом WebSocket, який встановлює двонаправлене повнодуплексне з'єднання між клієнтом та сервером [16]. Враховуючи, що стандартний Python-інтерфейс WSGI спроектований як суворо синхронний механізм обробки [17] і не здатен ефективно утримувати тисячі відкритих TCP-з'єднань, у системі інтегровано специфікацію ASGI. Це дозволяє асинхронно керувати подіями мережевого вводу-виводу [18], гарантуючи стабільну роботу комунікаційного шару навіть за умови одночасного проведення поєдинків на кількох татамі.

На основі проведеного архітектурного аналізу сформовано узагальнену структурну схему розроблюваного вебсервісу (рис. 1.4). Вона наочно демонструє розподіл відповідальності між клієнтським додатком, шлюзами синхронної та асинхронної обробки запитів, логічно ізольованими бізнес-модулями та рівнем персистентного зберігання даних.

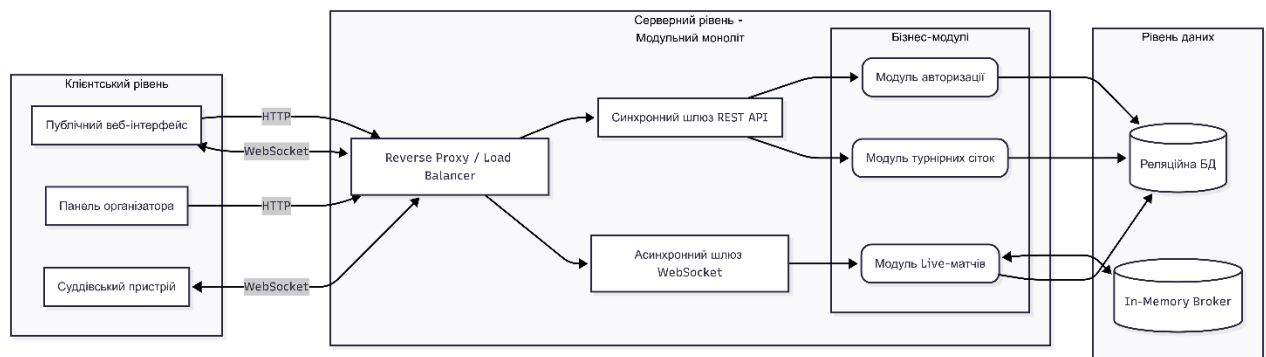


Рис. 1.4. Архітектура вебсервісу управління турнірними змаганнями

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		15

1.4. Обґрунтування вибору інструментальних засобів та вимоги до апаратного забезпечення

Вибір технологічного стеку для розробки високонавантаженої системи управління спортивними турнірами обумовлений двома критичними архітектурними вимогами: необхідністю реалізації складної реляційної бізнес-логіки (генерація рекурсивних сіток) та потребою у забезпеченні двонаправленої синхронізації даних у режимі реального часу (WebSockets). Для прийняття обґрунтованих інженерних рішень проведено порівняльний аналіз сучасних платформ, мов програмування, систем керування базами даних та клієнтських фреймворків.

Вибір мови програмування та серверного фреймворку. Для реалізації серверної частини (Backend) розглядалися три домінуючі на ринку екосистеми: Node.js (JavaScript/TypeScript), Go та Python. Платформа Node.js історично демонструє високі показники в обробці асинхронного вводу-виводу завдяки архітектурі Event Loop, проте її однопотокова природа стає вузьким місцем при виконанні ресурсоемних математичних операцій, таких як алгоритмічне балансування турнірних дерев. Мова Go пропонує найвищу продуктивність та низьке споживання оперативної пам'яті завдяки компіляції в машинний код та легковаговим горутинам, але вимагає значних витрат часу на розробку (Time-to-Market) базового функціоналу управління користувачами та доступом.

В якості основної мови обрано Python версії 3.13. Вирішальним фактором стала інтеграція нововведень згідно з PEP 703, що робить використання Global Interpreter Lock (GIL) опціональним, та PEP 744, який вводить базовий JIT-компілятор [19]. Це нівелює історичні проблеми Python із паралельним виконанням потоків, дозволяючи повноцінно утилізувати багатоядерні процесори при розрахунку алгоритмів жеребкування.

У контексті мови Python порівнювалися мікрофреймворки (FastAPI, Flask) та комплексні рішення (Django). Вибір зупинено на фреймворку Django версії 5.2 LTS

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		16

(довгострокова підтримка). На відміну від FastAPI, який вимагає ручного збирання архітектури зі сторонніх бібліотек (SQLAlchemy, Alembic, Pydantic), Django надає інтегрований інструментарій. Зокрема, наявність вбудованого об'єктно-реляційного відображення (ORM), системи міграцій та гото5вої панелі адміністратора кардинально прискорює реалізацію CRUD-операцій. Версія 5.2 LTS також запровадила нативну підтримку композитних первинних ключів (models.CompositePrimaryKey), що є критичним для проектування таблиць зв'язків «спортсмен–матч–категорія» без штучних сурогатних ідентифікаторів [20]. Для реалізації REST-інтерфейсів використовується бібліотека Django REST Framework 3.17, яка забезпечує строгу типізацію DTO-об'єктів (Data Transfer Objects) та автоматичну генерацію специфікацій OpenAPI 3 [21].

Детальний порівняльний аналіз серверних технологій наведено у таблиці 1.3.

Таблиця 1.3

Порівняльний аналіз серверних технологій для розробки бекенду

Критерій порівняння	Python 3.13 + Django 5.2	Node.js 22 + Express	Go 1.22 + Gin
Модель конкурентності	Багатопотоковість (опціональний GIL), ASGI для асинхронності	Однопотоковий Event Loop (Worker Threads для важких задач)	Легковагові горутини (M:N планувальник)
Швидкість розробки (Time-to-Market)	Дуже висока (вбудовані ORM, Auth, Admin)	Середня (потребує збірки зі сторонніх пакетів, напр., Prisma)	Низька (високий рівень деталізації коду, ручна обробка помилок)
Продуктивність (CPU-bound задачі)	Висока (завдяки JIT-компіляції та C-розширенням)	Низька (блокування Event Loop при математичних розрахунках)	Дуже висока (компіляція в нативний машинний код)
Архітектурна цілісність	Строго регламентована фреймворком (MVT)	Високо варіативна (залежить від рішень розробника)	Варіативна (відсутність єдиного стандарту структури)
Екосистема для обробки графів (сіток)	Розширена спільнота Data Science, готові алгоритмічні бібліотеки	Обмежена базовими структурами даних	Вимагає написання власних реалізацій структур даних

Вибір системи керування базами даних. Дані турнірних змагань характеризуються високим рівнем нормалізації: регламенти, клуби, тренери, спортсмени та матчі утворюють складну мережу зв'язків. Вибір СКБД здійснювався між реляційними (PostgreSQL, MySQL) та нереляційними (MongoDB) рішеннями. Використання NoSQL-орієнтованої MongoDB відхилено через відсутність надійної підтримки ACID-транзакцій для розподілених записів та неможливість ефективно виконувати JOIN-операції, необхідні для формування стартових протоколів.

Серед реляційних баз даних обрано PostgreSQL версії 18.3. Ключовою інженерною перевагою над MySQL є потужна підтримка рекурсивних узагальнених табличних виразів (Recursive CTE через оператор WITH RECURSIVE). Цей механізм є безальтернативним інструментом для SQL-рівня при обході деревоподібної структури турнірної сітки (наприклад, форматів Single/Double Elimination), де глибина дерева динамічно залежить від кількості учасників [22]. Крім того, незважаючи на загальну реляційну структуру, система вимагає зберігання неструктурованих конфігурацій локальних турнірів. PostgreSQL 18 вирішує цю проблему через нативні типи даних JSONB, що підтримують GIN-індексування та дозволяють виконувати швидкі пошукові запити безпосередньо по ключах JSON-документа, об'єднуючи переваги реляційного та документо-орієнтованого підходів.

Таблиця 1.4

Порівняльний аналіз систем керування базами даних

Функціональна характеристика	PostgreSQL 18.3	MySQL 9.0	MongoDB 7.0
Парадигма зберігання	Об'єктно-реляційна	Реляційна	Документо-орієнтована (NoSQL)
Відповідність вимогам ACID	Повна, суворі консистентність	Повна (у движку InnoDB)	Часткова (на рівні окремого документа/репліки)
Обробка ієрархічних даних	WITH RECURSIVE (висока оптимізація обходу графів)	Обмежена підтримка рекурсії	Відсутня (вимагає багаторазових запитів з коду програми)
Робота з JSON	Нативний JSONB тип із GIN/GiST індексами та операторами	Тип JSON без глибокої оптимізації пошуку	Основний формат зберігання (BSON)

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		18

Асинхронний I/O	Повна підтримка (io_uring), оптимізація конкурентних запитів	Обмежена	Присутня
-----------------	--	----------	----------

Комунікаційний шар реального часу. Для забезпечення миттєвої передачі результатів суддівства розглянуто технології Long Polling, Server-Sent Events (SSE) та WebSockets. Механізм SSE є однонаправленим (сервер–клієнт) і не дозволяє суддівським планшетам швидко надсилати бали без ініціації нового HTTP-запиту, що створює небажані мережеві накладні витрати. Тому використано протокол WebSocket. Оскільки ядро системи базується на Django, для обробки з'єднань використовується специфікація ASGI та пакет Django Channels 4.3.2. Цей пакет забезпечує маршрутизацію постійних з'єднань (AsyncWebsocketConsumer) паралельно із синхронними REST-запитами [23]. Роль брокера повідомлень (Channel Layer), що відповідає за розсилку подій (broadcasting) підключеним клієнтам у межах однієї групи (наприклад, конкретного змагального майданчика), виконує In-Memory сховище Redis версії 8.6, яке характеризується мінімальними затримками (sub-millisecond latency) [24].

Вибір технологій клієнтської частини. Розробка інтерфейсу користувача (Frontend) вимагає використання інструментарію, здатного ефективно керувати складним станом додатку (перемальовування турнірних сіток у реальному часі, обробка суддівських кліків). Аналізувалися фреймворки Angular, Vue 3 та бібліотека React. Angular пропонує занадто жорстку та надлишкову архітектуру для проєкту, що розробляється невеликою командою. Vue 3 має плавнішу криву навчання, проте поступається масштабами екосистеми.

Обрано бібліотеку React версії 19.2.1. Цей реліз запровадив нативну підтримку механізму Actions, що дозволяє обробляти асинхронні переходи та стани відправки форм (pending/error states) без використання громіздких сторонніх бібліотек типу Redux [25]. Збірка клієнтського додатку виконується не застарілим Webpack, а сучасним інструментом Vite 8.0, який використовує бандлер Rolldown, написаний на

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		19

Rust. Це забезпечує екстремально швидкий час «гарячого перезавантаження» (Hot Module Replacement) під час розробки та оптимізовану мініфікацію коду для продакшн-середовища [26].

Таблиця 1.5

Порівняльний аналіз клієнтських технологій (Frontend)

Характеристика	React 19 + Vite 8	Vue 3 + Vite 8	Angular 17 + Webpack
Архітектурний підхід	Компонентна бібліотека (UI-first)	Прогресивний фреймворк	Повноцінний монолітний фреймворк
Механізм оновлення DOM	Віртуальний DOM (Fiber), оптимізація рендеру через Actions	Віртуальний DOM із реактивною системою Proxy	Інкrementальний DOM, система Change Detection
Керування асинхронним станом	Вбудовані хуки use() та Actions (без Redux)	Composition API	RxJS (потребує глибокого розуміння реактивного програмування)
Інструмент збірки (Bundler)	Vite (Rust-based Rolldown, надшвидкий HMR)	Vite (Rust-based Rolldown)	Webpack / Angular CLI (повільніша збірка великих модулів)

Інструментарій розгортання та апаратні вимоги. Для усунення проблеми розбіжності середовищ ("працює на локальній машині") застосовано технологію контейнеризації Docker. Використовуються актуальні версії Docker Engine 29.4 та оркестратор локальних контейнерів Docker Compose v5, які дозволяють задекларувати всю інфраструктуру (БД, кеш, вебсервер) в єдиному конфігураційному файлі. У ролі зворотного проксі-сервера (Reverse Proxy) та балансувальника навантаження виступає Nginx 1.28.3, який маршрутизує HTTP-запити на WSGI-сервер (Gunicorn), а WebSocket-з'єднання – на ASGI-сервер (Daphne).

Апаратні вимоги до серверної частини (Host-сервер) для забезпечення стабільної роботи заявленого стеку технологій наведено у таблиці 1.6. Вимоги до клієнтської частини зводяться до наявності сучасного веб-оглядача (Chrome 120+, Safari 17+, Firefox 120+) із підтримкою специфікації ECMAScript 2023.

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		20

Рекомендовані вимоги до апаратного забезпечення серверної частини

Компонент інфраструктури	Мінімальні вимоги (Тестове середовище)	Рекомендовані вимоги (Продакшн)	Обґрунтування навантаження
Процесор (CPU)	2 ядра (x86_64 / ARM64)	4+ ядра	Паралельна робота JIT-компілятора Python 3.13 та обробка ASGI-воркерів
Оперативна пам'ять (RAM)	4 ГБ	8 ГБ	Резервація 2 ГБ під PostgreSQL, 1 ГБ під In-Memory Redis, решта – воркери Django
Дискова підсистема	20 ГБ SSD	50+ ГБ NVMe SSD	Забезпечення максимального IOPS для операцій реляційної бази даних
Мережевий інтерфейс	100 Мбіт/с	1 Гбіт/с	Утримання конкурентних WebSocket-з'єднань суддівських панелей

Висновки до першого розділу

У першому розділі пояснювальної записки проведено системний аналіз предметної області та обґрунтовано вибір архітектурних і технологічних рішень для розробки вебсервісу управління турнірними змаганнями.

Дослідження бізнес-процесів організації змагань із контактних видів спорту дозволило формалізувати вимоги до алгоритмів генерації турнірних сіток (Single Elimination, Double Elimination, Round Robin). Встановлено, що критичною інженерною вимогою до розроблюваної системи є забезпечення транзакційної цілісності бази даних при паралельній роботі суддівських бригад у режимі реального часу.

Порівняльний аналіз існуючих аналогів (MyUventex, Smoothcomp, Sportdata) виявив, що домінуючі на ринку рішення є закритими екосистемами, які обмежують можливості інтеграції та кастомізації під локальні регламенти. Це обґрунтувало доцільність створення власної системи на базі відкритої API-центричної архітектури.

Для програмної реалізації обрано архітектурний патерн «Модульний моноліт», що поєднує простоту розгортання зі строгою ізоляцією бізнес-логіки. Взаємодію між клієнтом та сервером розділено на синхронний (REST API) та асинхронний (WebSocket) канали комунікації. Як інструментальну базу затверджено екосистему Python (Django), реляційну СУБД PostgreSQL для виконання рекурсивних запитів турнірних дерев, бібліотеку React для клієнтської частини та технологію Docker для контейнеризації.

Сформовані вимоги, спроектована архітектура та обраний інженерний стек утворюють надійний фундамент для переходу до етапу проектування структури бази даних та розробки алгоритмічного ядра системи, що детально розглядається у другому розділі кваліфікаційної роботи.

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		22

РОЗДІЛ 2. ПРОЄКТУВАННЯ СИСТЕМИ

2.1. Визначення варіантів використання та об'єктно-орієнтованої структури системи

Процес розробки складних програмних комплексів для управління спортивними заходами вимагає системного підходу до аналізу вимог. Програмні вимоги є фундаментальною основою розробки, оскільки вони визначають функції, загальну поведінку та стандарти продуктивності майбутньої системи. Детальна специфікація цих вимог на ранніх етапах допомагає уникнути архітектурних невизначеностей та слугує надійною точкою відліку для створення надійного програмного продукту.

Аналіз функціональних та нефункціональних вимог. Функціональні вимоги зосереджуються на тому, як саме програмне забезпечення має працювати, і чітко визначають бажану поведінку системи у відповідь на дії користувача. Нефункціональні вимоги, на відміну від функціональних, орієнтовані на глобальну операційну діяльність системи, а не на її окремі функції, встановлюючи суворі обмеження щодо швидкості, безпеки та надійності роботи у реальному виробничому середовищі [27].

У контексті розроблюваної платформи функціональні вимоги охоплюють повний цикл адміністрування турнірних змагань. Система повинна забезпечувати автоматизацію процесів конфігурації вагових та вікових категорій, управління сіткою змагань за різними алгоритмами жеребкування та ведення електронних протоколів. Для організаторів важливою функціональною можливістю є оперативне редагування списків учасників та контроль регламенту. Для суддівського персоналу програмний продукт має надавати інтуїтивно зрозумілий інтерфейс для миттєвої фіксації подій безпосередньо на змагальному майданчику.

Однією з найважливіших нефункціональних вимог розроблюваної платформи є висока доступність та відмовостійкість сервісу, оскільки технічні збої під час активної

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		23

фази турніру є неприпустимими. Масштабованість бази даних повинна дозволяти обробляти значні масиви інформації при одночасному проведенні поєдинків на кількох майданчиках. Безпека даних гарантується через використання сучасних механізмів авторизації на основі токенів та криптографічне шифрування паролів.

Вимоги до часових характеристик реакції системи. Ефективність веб-орієнтованих систем реального часу безпосередньо залежить від затримок під час передачі даних між клієнтом та сервером. Для забезпечення оперативності та прозорості суддівства встановлюються жорсткі обмеження на швидкість відгуку програмного комплексу. Час реакції системи на фіксацію бала суддею та його відображення на публічному табло не повинен перевищувати двісті мілісекунд, що є критичним показником для динамічних видів спорту. Операції пошуку та фільтрації учасників у категоріях з великою кількістю заявок мають виконуватися протягом однієї секунди. Процес автоматичного формування турнірних дерев, що включає математичні розрахунки вільних проходів та розведення одноклубників по різних гілках сітки, повинен завершуватися не пізніше ніж за три секунди.

Специфікація варіантів використання системи. Діаграми прецедентів застосовуються для опису основних функцій системи та ідентифікації взаємодії між цією системою та її зовнішнім середовищем, яке моделюється у вигляді акторів [28]. У системі виділено п'ять ключових ролей користувачів, кожна з яких має специфічні права та бізнес-завдання. Організатор турніру відповідає за глобальне налаштування заходу та ініціалізацію жеребкування. Суддя безпосередньо керує поєдинком через планшетний інтерфейс. Представник команди здійснює масову реєстрацію атлетів. Глядачі мають доступ до моніторингу результатів у реальному часі. Адміністратор системи забезпечує управління всіма обліковими записами.

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		24

Таблиця 2.1 – Опис варіантів використання системи

Актор	Варіант використання	Формулювання дії
Організатор	Створення турніру	Визначення назви, дати, локації та базових правил змагань.
Організатор	Налаштування категорій	Формування вагових та вікових груп учасників згідно з регламентом.
Організатор	Генерація сіток	Запуск алгоритму автоматичного розподілу учасників по парах.
Тренер	Реєстрація атлетів	Подання заявок від спортивного клубу на участь у категоріях.
Суддя	Ведення поєдинку	Фіксація балів, попереджень та часу матчу через вебінтерфейс.
Суддя	Завершення матчу	Офіційне підтвердження результату та визначення переможця.
Глядач	Моніторинг сіток	Перегляд актуального прогресу турніру та турнірної таблиці.
Адміністратор	Управління ролями	Призначення прав доступу модераторам та організаторам.

Візуальне представлення взаємодії користувачів із функціональними блоками системи наведено на рисунку 2.1.

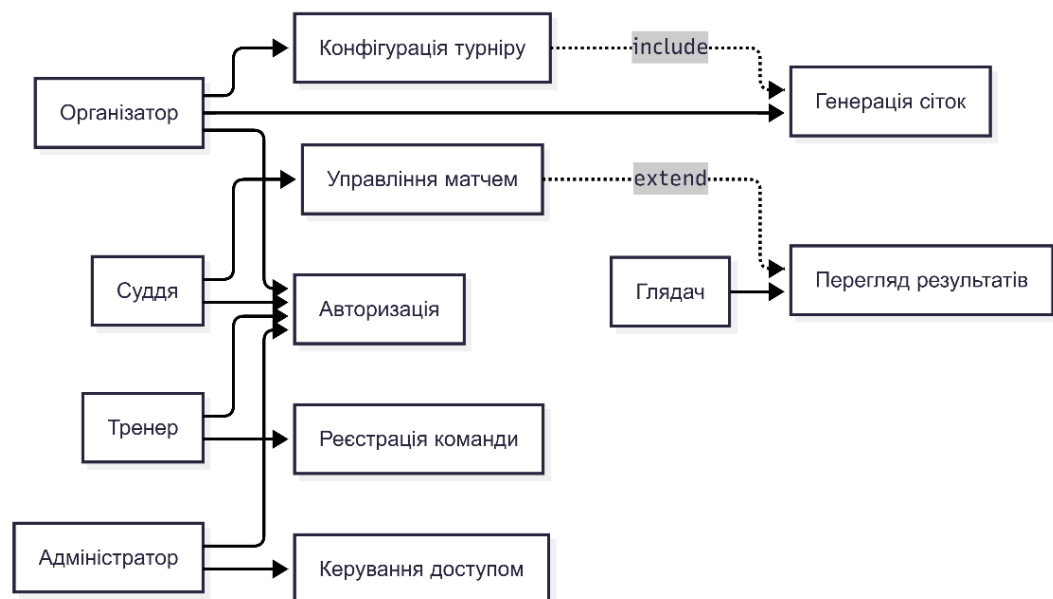


Рис. 2.1. Діаграма варіантів використання системи

Об'єктно-орієнтована структура системи. Проектування архітектури програмного комплексу базується на принципах модульного моноліту та патерну активного запису для взаємодії з реляційною базою даних. Кожен компонент системи інкапсулює частину бізнес-логіки та відображає фізичну структуру збереження даних. Для проектування та візуалізації такої архітектури використовується діаграма класів, яка моделює кожну окрему сутність програмного комплексу, деталізуючи типи атрибутів, методи та кардинальність об'єктних взаємозв'язків.

Для реалізації підсистеми управління користувачами застосовується інтегрована модель користувача базового фреймворку розробки. Вона містить стандартні поля ідентифікатора, електронної пошти, хешованого пароля та статусу активності, що дозволяє уникнути дублювання коду при написанні механізмів авторизації. Сутність турніру виступає кореневим об'єктом ієрархії змагань. Вона включає цілочисельний первинний ключ, текстове поле для назви, поле дати і часу початку заходу, а також рядкове поле статусу, яке забезпечує контроль життєвого циклу турніру від етапу збору заявок до повного завершення.

Деталізація структури змагань реалізується через сутність категорії, яка пов'язана з турніром відношенням композиції. Категорія визначає жорсткі межі для допуску спортсменів і містить числові поля мінімального та максимального віку, дробові поля меж ваги, а також логічний прапорець для визначення формату командної естафети. Сутність учасника є сполучною ланкою між профілем користувача та конкретною категорією. Вона зберігає зовнішні ключі на відповідні моделі та текстове поле назви спортивного клубу, що є критично важливим параметром для подальшої роботи алгоритму розведення одноклубників під час жеребкування.

Найвищий рівень складності зосереджено в сутності матчу, яка формує орієнтований граф поєдинків. Модель містить посилання на категорію, зовнішні ключі на першого та другого учасників, цілочисельні індекси раунду та порядкового

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						26
Змн.	Арк.	№ докум.	Підпис	Дата		

Бізнес-логіка генерації сіток інкапсульована в окремому сервісному класі генератора. Його головний метод реалізує побудову олімпійської системи на вибування. Алгоритм послідовно отримує список усіх верифікованих учасників обраної категорії, обчислює найближчий більший ступінь двійки для визначення розмірності сітки, розраховує кількість порожніх місць та генерує необхідну кількість записів матчів у базі даних, автоматично зв'язуючи їх рекурсивними ключами для формування цілісного дерева. Статичну структуру описаних сутностей та їх атрибутів наочно продемонстровано на рисунку 2.2.



2.2. Розробка бази даних системи

Проектування фізичного рівня збереження даних є фундаментальним етапом перетворення розробленої об'єктно-орієнтованої моделі у конкретну реляційну структуру. Основною метою цього етапу є створення цілісного, відмовостійкого та продуктивного середовища для збереження інформації, що забезпечує стабільну роботу системи під час пікових навантажень. Фізична модель даних розробляється з урахуванням принципів нормалізації до третьої нормальної форми, що дозволяє усунути надмірність даних, запобігти виникненню аномалій при оновленні записів та гарантувати транзакційну цілісність при одночасному доступі кількох суддівських бригад. У межах проектування особлива увага приділяється реалізації зв'язків між сутностями, визначенню суворих типів даних та налаштуванню обмежень на рівні зовнішніх ключів для автоматизації контролю консистентності інформації

Нижче наведено детальну специфікацію структури збереження даних, що включає опис усіх таблиць програмного комплексу, їх атрибутів, типів даних та функціонального призначення (див. Додаток А).

Таблиця 2.2 - Специфікація таблиці спортивних клубів (club)

Назва поля	Тип даних	Призначення
id	integer	Первинний ключ, унікальний ідентифікатор клубу.
name	varchar(200)	Офіційна назва спортивної організації або федерації.
region	varchar(150)	Географічна приналежність клубу (область, місто).

Таблиця спортивних клубів є базовою довідковою сутністю, яка забезпечує ієрархічну структурованість даних у системі. Вона відіграє критичну роль у функціонуванні алгоритмів автоматичного жеребкування, оскільки дозволяє ідентифікувати приналежність атлета до конкретної організації. Це необхідно для реалізації вимоги розведення одноклубників по різних гілках турнірної сітки на початкових етапах змагань, що гарантує об'єктивність результатів та запобігає передчасним поєдинкам між представниками однієї команди.

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						28
Змн.	Арк.	№ докум.	Підпис	Дата		

Таблиця 2.3 — Специфікація таблиці користувачів (auth_user)

Назва поля	Тип даних	Призначення
id	integer	Первинний ключ, ідентифікатор облікового запису.
email	varchar(254)	Електронна пошта, що використовується як логін.
password_hash	varchar(128)	Криптографічний хеш пароля користувача.
first_name	varchar(150)	Ім'я користувача.
last_name	varchar(150)	Прізвище користувача.
role	varchar(50)	Системна роль (Організатор, Тренер, Суддя).
club_id	integer	Зовнішній ключ на таблицю клубів (може бути порожнім).

Дана таблиця інтегрує механізми автентифікації та авторизації системи. Використання поля для збереження криптографічного хешу замість відкритого тексту забезпечує відповідність сучасним стандартам безпеки програмного забезпечення [29]. Поле ролі є визначальним для реалізації моделі контролю доступу, що обмежує можливості користувачів відповідно до їх повноважень (наприклад, заборона тренерам редагувати протоколи поєдинків). Зв'язок із таблицею клубів дозволяє автоматично асоціювати тренера з його організацією, що спрощує процес масової реєстрації підопічних.

Таблиця 2.4 — Специфікація таблиці спортсменів (athlete)

Назва поля	Тип даних	Призначення
id	integer	Первинний ключ профілю спортсмена.
coach_id	integer	Зовнішній ключ на таблицю користувачів (відповідальний тренер).
club_id	integer	Зовнішній ключ на таблицю спортивних клубів.
first_name	varchar(100)	Ім'я спортсмена.
last_name	varchar(100)	Прізвище спортсмена.
gender	varchar(10)	Стать атлета (для фільтрації категорій).
birth_date	date	Повна дата народження атлета.
base_weight	numeric(5,2)	Базова вага, зафіксована у загальному профілі.

Продовження таблиці 2.4.

skill_level	varchar(50)	Кваліфікаційний рівень або колір пояса (необов'язкове).
-------------	-------------	---

Сутність спортсмена відокремлена від системних облікових записів, що дозволяє реалізувати гнучке управління даними без необхідності реєстрації кожного атлета як користувача системи. Це відповідає реальним потребам спортивних секцій, де адміністративні дії виконує представник команди. Наявність полів статі та дати народження дозволяє системі автоматично пропонувати лише ті категорії турніру, вимогам яких відповідає конкретний спортсмен. Зовнішній ключ на тренера (coach_id) визначає право власності на дані та можливість їх редагування.

Таблиця 2.5 — Специфікація таблиці турнірів (tournament)

Назва поля	Тип даних	Призначення
id	integer	Первинний ключ турніру.
organizer_id	integer	Зовнішній ключ на організатора змагань.
title	varchar(255)	Повна назва заходу.
sport_type	varchar(100)	Вид єдиноборства (Карате, Дзюдо, ММА тощо).
location	varchar(255)	Місце проведення (місто, арена).
start_date	timestamp	Дата та час початку офіційної частини.
end_date	timestamp	Дата та час офіційного завершення змагань.
status	varchar(20)	Поточний стан (Draft, Registration, Active, Completed).

Таблиця турнірів є центральним об'єктом організаційного рівня. Введення поля виду спорту дозволяє системі адаптувати бізнес-правила (наприклад, доступні методи завершення матчу) під специфіку конкретного єдиноборства. Поля дати початку та завершення забезпечують коректне планування та формування архіву змагань. Статус турніру використовується логікою бекенд-частини для керування доступністю функцій реєстрації та автоматичної генерації звітних протоколів після завершення останнього поєдинку.

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		30

Таблиця 2.6 — Специфікація таблиці категорій (category)

Назва поля	Тип даних	Призначення
id	integer	Первинний ключ категорії.
tournament_id	integer	Зовнішній ключ на відповідний турнір.
name	varchar(100)	Назва категорії (наприклад, «Юніори, до 75 кг»).
allowed_gender	varchar(10)	Гендерні обмеження для учасників.
min_age	integer	Мінімальний поріг віку учасника.
max_age	integer	Максимально допустимий вік.
min_weight	numeric(5,2)	Нижня межа вагової категорії.
max_weight	numeric(5,2)	Верхня межа вагової категорії.
allowed_skill_level	varchar(50)	Допустимий рівень майстерності (необов'язкове).
bracket_format	varchar(50)	Тип турнірної системи (Олімпійська, Кругова тощо).

У цій таблиці зберігаються регламентні обмеження, що визначають структуру змагань. Вибір формату турнірної сітки на рівні категорії дозволяє проводити один і той самий турнір за різними правилами для різних груп учасників (наприклад, кругова система для малих груп та олімпійська для масових категорій). Поля вікових та вагових меж використовуються для валідації заявок на етапі реєстрації та під час проходження мандатної комісії.

Таблиця 2.7 - Специфікація таблиці реєстрацій (registration)

Назва поля	Тип даних	Призначення
id	integer	Первинний ключ заявки на участь.
athlete_id	integer	Зовнішній ключ на профіль спортсмена.
category_id	integer	Зовнішній ключ на обрану категорію.
seed_number	integer	Номер учасника для жеребкування за рейтингом.
recorded_weight	numeric(5,2)	Фактична вага спортсмена на момент зважування.
status	varchar(20)	Поточний стан заявки (обов'язкове).

Таблиця реєстрації реалізує відношення багато-до-багатьох між атлетами та категоріями, дозволяючи фіксувати стан учасника безпосередньо в контексті

конкретного заходу. Поле фактичної ваги (recorded_weight) є критично важливим, оскільки воно зберігає результат офіційного зважування, який може відрізнитися від базової ваги у профілі атлета. Ці дані є юридичною основою для допуску до поєдинків та формування підсумкових протоколів мандатної комісії.

Таблиця 2.8 — Специфікація таблиці матчів (match)

Назва поля	Тип даних	Призначення
id	integer	Первинний ключ поєдинку.
category_id	integer	Зовнішній ключ на відповідну категорію.
reg_first_id	integer	Зовнішній ключ на реєстрацію першого учасника.
reg_second_id	integer	Зовнішній ключ на реєстрацію другого учасника.
round_index	integer	Номер етапу (кола) турнірної сітки.
match_order	integer	Порядковий номер матчу на етапі.
tatami_number	integer	Номер змагального майданчика.
score_first	integer	Поточна кількість балів першого учасника.
score_second	integer	Поточна кількість балів другого учасника.
warnings_first	integer	Кількість штрафних попереджень першого бійця.
warnings_second	integer	Кількість штрафних попереджень другого бійця.
winner_id	integer	Зовнішній ключ на реєстрацію переможця.
loser_next_match_id	integer	Ключ на наступний поєдинок того, хто програв (необов'язкове).
match_duration	integer	Тривалість поєдинку в секундах (необов'язкове).
win_method	varchar(100)	Офіційний спосіб здобуття перемоги.
next_match_id	integer	Рекурсивний ключ на наступний поєдинок у сітці.
status	varchar(20)	Стан матчу (Scheduled, Ongoing, Completed).

Дана сутність є найбільш технічно складною у базі даних. Вона забезпечує повний моніторинг ходу змагань у реальному часі.

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		32

Використання рекурсивного зв'язку (next_match_id) дозволяє автоматично маршрутизувати переможця по турнірному дереву без втручання організатора. Поля балів та попереджень забезпечують синхронізацію публічних табло із діями суддівської бригади. Наявність поля способу перемоги та ідентифікатора переможця дозволяє автоматично формувати підсумкову статистику змагань та обчислювати рейтинги спортсменів після завершення турніру.

Забезпечення цілісності та продуктивності бази даних

Розроблена фізична структура бази даних забезпечує високий рівень надійності та стійкості до помилок під час паралельної обробки великих масивів інформації. Дотримання третьої нормальної форми дозволяє усунути надмірність даних, що не лише мінімізує обсяги дискового простору, але й гарантує відсутність логічних суперечностей при оновленні результатів змагань. Використання механізмів зовнішніх ключів на рівні системи керування базами даних PostgreSQL забезпечує автоматичний контроль цілісності: видалення або зміна батьківських записів (наприклад, видалення турніру) супроводжується каскадними діями або обмеженнями, що запобігають появі некоректних посилань у системі.

Для підвищення продуктивності при формуванні турнірних сіток та відображенні результатів у реальному часі передбачено створення індексів за ключовими полями, такими як ідентифікатори категорій та учасників. Це дозволяє скоротити час виконання складних рекурсивних запитів до таблиці матчів. Обрана архітектура зв'язків, що розділяє облікові записи користувачів та профілі атлетів, надає системі необхідну гнучкість для масштабування та підтримки різних видів одноборств без необхідності структурної перебудови бази даних. Таким чином, спроектована модель є надійним фундаментом для програмної реалізації бізнес-логіки турнірного менеджменту.

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		33

ВИСНОВКИ

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		34

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

[1] Hoyer, R. et al. «Reimagining Sports Businesses in the Digital Realm: Exploring Transformative Indicators in the Era of Virtualization». Preprints.org, 2026. *Дослівна цитата з джерела:* «Digitization in sports integrates administrative functions such as player registration, ranking lists, tournament management, and results recording into a comprehensive ICT system.»

[2] World Karate Federation. «WKF Kata and Kumite Competition Rules». Офіційний портал WKF, 2024. URL: <https://www.wkf.net/pdf/rules/wkf-competition-rules-2024.pdf> *Дослівна цитата з джерела:* «For Team matches [...] The Coach must present the team lineup to the official table. The team must consist of athletes according to the categories and genders established for the competition.»

[3] BracketsNinja. «Single-Elimination Brackets: Ultimate Tournament Guide». BracketsNinja Blog, 2024. URL: <https://www.bracketsninja.com/types/single-elimination-bracket> *Дослівна цитата з джерела:* «When the number of participants isn't a perfect power of two (4, 8, 16, 32, 64, etc.), the bracket incorporates "byes" in the first round. A bye allows certain participants to skip the opening round and automatically advance.»

[4] Stream. «Real-time Communication with WebSockets». GetStream.io Fundamentals, 2024. URL: <https://getstream.io/resources/projects/web rtc/fundamentals/real-time-communication-with-websockets/> *Дослівна цитата з джерела:* «Unlike HTTP, which follows a request-response pattern, WebSockets allow for real-time, two-way communication. [...] WebSockets excel in applications requiring real-time updates or low-latency communication: Live sports updates: Scores and plays update in real-time.»

[5] Challonge. «Double Elimination Tournament Format Guide». Challonge Knowledge Base, 2025. URL: <https://challonge.com/guides/tournament-formats/double-elimination> *Дослівна цитата з джерела:* «Double elimination consists of two brackets: a winner's bracket and a loser's bracket. If a participant loses in the winner's bracket, they are moved to the loser's bracket where they still have a chance to compete for a third-place finish or even the championship.»

[6] SportsRec. «Rules for a Round Robin Tournament». SportsRec Management, 2024. URL: <https://www.sportsrec.com/6692797/round-robin-tournament-rules> *Дослівна цитата з джерела:* «In a round robin tournament, every participant plays every other participant in their group once. This format is ideal for smaller groups to ensure a fair ranking based on total wins and points accumulated.»

[7] Wikipedia (English). «Seed (sports)». Wikipedia, The Free Encyclopedia, 2026. URL: [https://en.wikipedia.org/wiki/Seed_\(sports\)](https://en.wikipedia.org/wiki/Seed_(sports)) *Дослівна цитата з джерела:* «Seeding is the preliminary ranking of players or teams for the purpose of draw. Organizers use seeding to ensure that top-ranked participants or participants from the same organization do not face each other until the late stages of the tournament.»

[8] Платформа MyUventex. Офіційний сайт. URL: <https://uventex.com/>

[9] Платформа Smoothcomp. Офіційний сайт. URL: <https://smoothcomp.com/>

Арк.

ІПЗ.КР.Б – 121 – 26 – ПЗ

35

Змн.	Арк.	№ докум.	Підпис	Дата

[10] Платформа Sportdata. Офіційний сайт. URL: <https://www.sportdata.org/>

[11] Lewis J., Fowler M. «Microservices». *MartinFowler.com*, 2014. URL: <https://martinfowler.com/articles/microservices.html> Дослівна цитата з джерела: «The microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API.»

[12] Fowler M. «MonolithFirst». *MartinFowler.com*, 2015. URL: <https://martinfowler.com/bliki/MonolithFirst.html> Дослівна цитата з джерела: «Almost all the successful microservice stories have started with a monolith that got too big and was broken up. Almost all the cases where a system that was built as a microservice system from scratch, it has ended up in serious trouble.»

[13] Grzybek K. «Modular Monolith: A Primer». *KamilGrzybek.com*, 2019. URL: <https://www.kamilgrzybek.com/blog/series/modular-monolith/> Дослівна цитата з джерела: «An architectural pattern where the system is composed of independent, self-contained modules, forming a single deployment unit, where modules interact only through well-defined interfaces.»

[14] Heinemeier Hansson D. «The Majestic Monolith». *Signal v. Noise*, 2016. URL: <https://signalvnoise.com/svn3/the-majestic-monolith/> Дослівна цитата з джерела: «The monolith model has been responsible for some of the biggest applications on the web. It's an architecture that consciously refuses unnecessary abstractions and distribution.»

[15] Fielding R. T. «Architectural Styles and the Design of Network-based Software Architectures». Ph.D. dissertation, University of California, Irvine, 2000. URL: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm> Дослівна цитата з джерела: «Separation of concerns is the principle behind the client-server constraints. A proper separation of functionality should simplify the server component in order to improve scalability.»

[16] Fette I., Melnikov A. «The WebSocket Protocol. RFC 6455». *Internet Engineering Task Force (IETF)*, 2011. URL: <https://datatracker.ietf.org/doc/html/rfc6455> Дослівна цитата з джерела: «The WebSocket Protocol enables two-way communication between a client running untrusted code in a controlled environment to a remote host [...] The protocol consists of an opening handshake followed by basic message framing, layered over TCP.»

[17] Eby P. J. «PEP 3333 – Python Web Server Gateway Interface v1.0.1». *Python Software Foundation*, 2010. URL: <https://peps.python.org/pep-3333/> Дослівна цитата з джерела: «This PEP proposes a simple and universal interface between web servers and web applications or frameworks.»

[18] ASGI Documentation. «ASGI (Asynchronous Server Gateway Interface)». *Read the Docs*. URL: <https://asgi.readthedocs.io/en/latest/> Дослівна цитата з джерела: «The WSGI specification [...] is irrevocably tied to the HTTP-style request/response cycle. ASGI is designed to provide a standard interface between async-capable Python web servers, frameworks, and applications, supporting WebSocket and other protocols.»

[19] Python Software Foundation. «What's New In Python 3.13». *Python.org*, 2026. URL: <https://docs.python.org/3/whatsnew/3.13.html> Дослівна цитата з джерела: «Python 3.13 includes

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		36

an experimental free-threaded build (PEP 703) and a preliminary JIT compiler (PEP 744), laying the groundwork for significant performance improvements in multi-threaded workloads.»

[20] Django Software Foundation. «Django 5.2 release notes». *djangoproject.com*, 2025. URL: <https://docs.djangoproject.com/en/5.2/releases/5.2/> Дослівна цитата з джерела: «Django 5.2 introduces support for composite primary keys (models.CompositePrimaryKey), resolving a long-standing request for better integration with legacy databases and complex relational models.»

[21] Encode OSS. «Django REST Framework 3.17 Release». *django-rest-framework.org*, 2026. URL: <https://www.django-rest-framework.org/community/release-notes/> Дослівна цитата з джерела: «Version 3.17 removes legacy coreapi support, focusing entirely on OpenAPI 3 schema generation, and provides official compatibility with modern Python 3.14 features.»

[22] PostgreSQL Global Development Group. «PostgreSQL 18 Documentation: WITH Queries (Common Table Expressions)». *postgresql.org*, 2026. URL: <https://www.postgresql.org/docs/18/queries-with.html> Дослівна цитата з джерела: «WITH RECURSIVE provides a way to refer to the query's own output, which is essential for evaluating hierarchical data or tree structures directly in the database.»

[23] Django Channels Documentation. «Introduction to Channels 4.3». *channels.readthedocs.io*, 2025. URL: <https://channels.readthedocs.io/en/latest/introduction.html> Дослівна цитата з джерела: «Channels builds upon the native ASGI support in Django to handle WebSockets, chat protocols, and IoT protocols, routing long-lived connections alongside standard HTTP views.»

[24] Redis Ltd. «Redis 8.6 Release Notes». *redis.io*, 2026. URL: <https://redis.io/docs/latest/release-notes/> Дослівна цитата з джерела: «Redis 8 introduces improved asynchronous I/O and reduced latency for Pub/Sub operations, making it the ideal message broker for large-scale deployments.»

[25] Meta Platforms. «React 19 Release Blog». *react.dev*, 2024. URL: <https://react.dev/blog/2024/12/05/react-19> Дослівна цитата з джерела: «React 19 introduces Actions (support for async functions in transitions) with automatic pending and error states, significantly simplifying data mutation workflows.»

[26] Vite Core Team. «Vite 8.0: The Rolldown Era». *vite.dev*, 2026. URL: <https://vite.dev/blog/announcing-vite-8> Дослівна цитата з джерела: «Vite 8 ships with Rolldown as its single, unified, Rust-based bundler, delivering up to 10-30x faster builds while maintaining full plugin compatibility.»

Jama Software. "Functional vs Nonfunctional Requirements: What's the difference?". *Jama Software Blog*, 2021. URL: <https://www.jamasoftware.com/blog/requirements-management-functional-requirement-vs-non-functional-requirements/> Дослівна цитата з джерела: «Functional requirements focus on how the software must perform and specify the desired behavior of the system... Non-functional requirements are focused on the system's operation rather than its behaviors...»

IBM Documentation. "Creating use-case diagrams". *IBM Engineering Lifecycle Management*, 2023. URL: <https://www.ibm.com/docs/en/rational-soft-arch/9.6.1?topic=diagrams-creating-use-case> Дослівна цитата з джерела: «Use-case diagrams describe the main functions of a system and identify the interactions between the system and its external environment, represented by actors.»

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		37

OWASP Foundation. «Password Storage Cheat Sheet». *OWASP Guidance*, 2025. URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html Дослівна цитата з джерела: «Passwords must never be stored in plain text. Use a cryptographically strong, salted, and iterative hashing algorithm to store user credentials safely.»

ДОДАТКИ

		Муравицький В. Ю.			ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		38

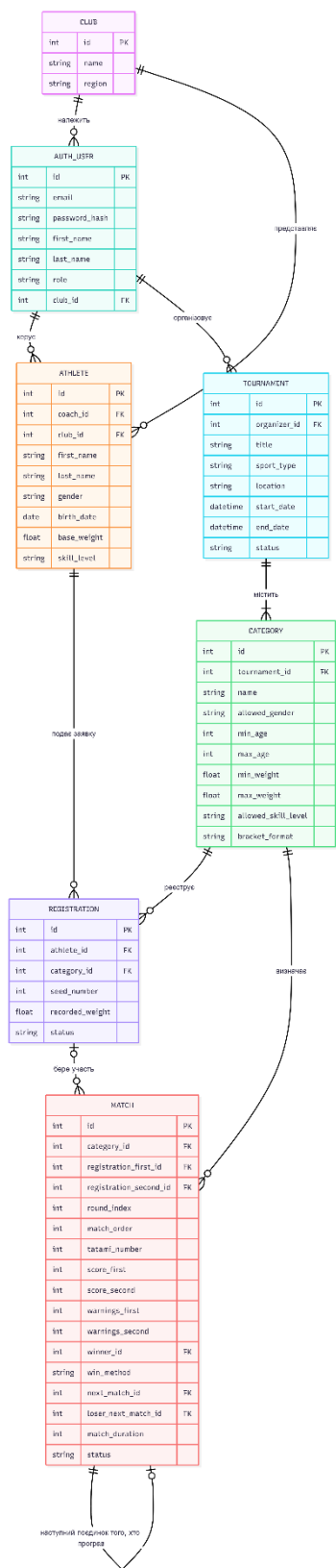


Рис. А.1. Схема зв’язків сутностей бази даних (ER-діаграма)